

Optional

Definición:

Optional es la herramienta estándar para representar la posible ausencia de valor (puede tener algo o puede no tener nada) evitando el típico error de **NullPointerException** al evitar el uso de null como retorno de métodos.

Forma de uso:

- Tenemos que pensar en el optional, a modo de **caja** (Optional< t >).

Imagina que pides un paquete por Amazon, el optional nos dice que la caja que nos llega puede tener nuestro pedido o estar vacía, pero antes de conocerlo, la caja puede que nos llegue o puede que NO nos llegue (es un null).

Esta es la principal diferencia, con optional nos aseguramos que nos llega la **caja**, aunque esta esté vacía. Evitando el **NullPointerException**.

- En lugar de devolver un objeto que podría ser un **null**, devuelves la **caja**, y tú como programador tienes que mirar primero qué hay en esa caja antes de usar lo que tenga dentro.
- **!!!IMPORTANTE!!!** Si tu creas un método que te promete en su cabecera que devuelve un Optional, '**public Optional<Alumno> buscarAlumno()**' y después dentro cuando no encuentras al Alumno haces '**return null**' **nos cargamos todo y el programa peta.**
Lo correcto es hacer '**return Optional.empty()**'. De esta forma, entregas la "caja vacía" y el programa no peta.

Métodos más relevantes:

.ofNullable (T valor): El más importante de todos:

1. Crea la caja.
2. Si el valor **existe**, lo **guarda**. Si no existe devuelve la caja vacía (vacía que no es lo mismo que null) evitando el “maldito” **NullPointerException**.

```
// Metemos el alumno en la caja y si es null, la caja estará vacía
Optional<Alumno> cajaAlumno = Optional.ofNullable(alumno);
```

.isPresent ():

Nos devuelve un **true** si al mirar dentro de la **caja** hay algo, o un **false** si dentro no hay nada.

```
// Si dentro de la caja hay un alumno, entra en el if, sino pasará al else.
if (cajaAlumno.isPresent()) {
    System.out.println("El alumno está presente.");
} else {
    System.out.println("El alumno no está presente.");
}
```

.ifPresent (Consumer):

Si hay algo dentro de nuestra caja, haz esto otro. Aquí se introduce un concepto llamado lambda que nos explicaran otros compañeros.

```
// Si dentro de la caja hay un alumno, lo muestra.
cajaAlumno.ifPresent(a -> {
    System.out.println(a.toString());
});
```

.orElse (T otro):

Este es el plan B, en el que decimos: si en esta caja no hay nada, entonces dame esto **otro**.

```
// Sacamos el texto de esa nueva caja. Si está vacía, usamos el Plan B con .orElse(), que nos devuelve "name"
String nombre = cajaNombre.orElse(name);
```

.orElseThrow ():

Si la caja está vacía, lanza un error (**NoSuchElementException**). Se usa si queremos que el programa pare en caso de que un dato no exista, siendo un fallo controlado por nosotros a diferencia del **NullPointerException**.

```
//Si la caja está vacía, nos mostrará un error. Si tiene algo, simplemente pasa de largo.  
Alumno alumno = cajaAlumno.orElseThrow(() ->  
    new NoSuchElementException("Error: No se puede modificar. El alumno no existe.")  
);
```

.map (Function mapper): De más nivel pero muy útil:

Si hay algo dentro, lo transforma.

Puedes pasar de un '**Optional<Alumno>**' a un '**Optional<String>**' con el nombre del Alumno, y en caso de que estuviese vacío, simplemente no hace nada, lo ignora.

```
// El .map() mira en la caja y si tiene al alumno, coge su nombre y lo mete en otra caja.  
//Si no hay nada devuelve la caja vacía.  
Optional<String> cajaNombre = cajaAlumno.map(a -> a.getNombre());
```

.empty (): La mencionamos antes en "Forma de uso"

Devuelve la "caja" vacía en lugar de un null que nos pite el programa.

```
for ( ){  
    }  
    // Si el bucle termina sin encontrar coincidencias, devolvemos la caja vacía.  
    return Optional.empty();
```

Ejemplo en Java

Crear un programa para gestionar los alumnos de 1º de DAM. El objetivo principal es implementar las operaciones CRUD clásicas y algunas operaciones extra eliminando por completo el uso de **null** en los retornos mediante la clase **Optional**.

Clase Alumno: Debe tener dni , nombre y notaMedia. Sus getters, setters y toString().

Clase GestorAlumnos: Debe contener un ArrayList con Alumnos y los métodos CRUD usando exclusivamente las herramientas vistas en teoría.

Además crear un método para modificar la notaMedia usando .orElseThrow() y otro para obtener el nombre de un alumno pasado como parámetro mediante el uso de .map() y .orElse().